

Enzyme BO benchmark: session report

Two bugs, a corrected criterion, and a box that moves the other way

10 August 2026

Contents

What this is	1
The benchmark, briefly	2
Two bugs came first	2
The kernel, benchmarked for the first time	2
The criterion is a ratio, not a curve	3
The catch	5
What I retracted	5
What changed in the tree	5
A note on fixed design variables	6
What I would do next	6
Where things are	7

What this is

You pointed me at the `enzyme-gbdt` worktree and its `docs/next-agent.md`. That file carries a standing task from two earlier sessions:

Tune the problem such that the iteration advantage (nearly $2\times$ for meta) converts into final metrics. We need a clean tradeoff: a smooth problem that benefits from iterations.

You later widened the scope: the design space, `n_experiments`, the objective, the system itself and the enzyme parameters are all in play, provided they stay realistic. This report assumes you have read nothing else.

Status: the task is not finished. Two bugs had to be fixed before any tuning could be judged, the task's criterion turned out to be a different quantity than "best-so-far keeps falling", and the one config change that follows from the measurements reverses last session's decision. Everything here is the GBDT proxy objective; nothing has been run on the neural objective.

The benchmark, briefly

A **design** is a batch of four experiments, each an (enzyme stock fraction, temperature) pair — 8 numbers. Each experiment simulates a hexokinase reaction for an hour and returns 8 noisy read-outs. A regressor infers the enzyme’s optimal operating temperature T^* from the batch; the converged loss of that inference is what Bayesian optimisation minimises over designs.

For this study the regressor is a gradient-boosted tree, not the neural meta-regressor: ~ 0.7 s per design against 27–137 s, which is what makes 120-iteration campaigns affordable. Losses are normalised by the T_{melting} prior, so every table also gives **°C of target resolution** ($\sqrt{\text{loss}} \times \text{prior half-range}$), which stays comparable across changes that move the normaliser.

Read-out noise is the one parameter worth stating precisely, because it is quoted throughout. `measurement_noise` is an ABSOLUTE Gaussian standard deviation in mM, added independently to each read-out: `measurements = concentration + measurement_noise * normal()`. The default $\sigma = 0.05$ mM sits against a 1 mM stock that the design dilutes — $A_0 = \text{concentration_A} \cdot (1 - \text{fraction})/2 \leq 0.5$ mM — so σ is 10% of the initial substrate at zero enzyme fraction, 15% at $\frac{1}{3}$, 20% at $\frac{1}{2}$, and worse still on the later read-outs as $[A]$ falls. It is emphatically not “5% noise”.

The **null** everything is measured against is random search drawn from the same box, scored on the same function, with the same number of evaluations.

Two bugs came first

The invariant kernel had never fitted

The batch is a *set*: all 24 orderings of a design are one design with one loss. A previous session built `PermutationInvariantRBF` to put that symmetry in the GP kernel, computing the group average as a permanent via Ryser’s formula.

Its hyperparameter gradient was written as $P(S) \cdot \sum_i dR_i(S) / R_i(S)$. Those row sums are RBF values — positive in exact arithmetic, but they **underflow to exactly 0 in float64** at short lengthscales, which the prior bounds permit ($\log l$ down to -6 , i.e. $l = 0.0025$ against unit-cube distances) and which L-BFGS reaches on its own restarts. The gradient returned NaN, scikit-learn’s L-BFGS quit at iteration 0 with ABNORMAL, and the GP silently kept its **initial** hyperparameters. It never fitted, and it never said so.

Fixed by removing the division: $d/d\theta P(S) = \sum_i (\prod_{i' \neq i} R_{i'}) \cdot dR_i$, with the product-of-others from prefix/suffix cumulative products — exact when a row vanishes, same cost at these sizes. Verified finite from $l = 2.0$ down to $l = 0.0025$; the 15 existing kernel tests (finite differences, autodiff of an independent JAX re-implementation, Ryser against the permanent’s definition) pass.

Every pre-fix “invariant kernel” number is void.

The kernel could not be selected

The driver read its kernel from `config["enzyme"]["kernel"]`, but the detector config is validated against `__init__`, so that key raises unknown `EnzymeDetector config key` and can never be set. The read was dead; only the fallback ever ran. The kernel is now an explicit `--kernel` flag.

The kernel, benchmarked for the first time

8 BO seeds per kernel against 10 random-search instances, 60 iterations, one shared null, everything else identical. Settings: $T \in [35, 68]$ °C, 4 experiments, read-out noise $\sigma = 0.05$ mM, T_{melting} prior [45, 58].

arm	@20	@60	@60 °C	localisation	vs null
ARD-RBF (no symmetry)	0.0703	0.0499	1.45	0.84	p = 0.20
Permutation-invariant RBF	0.0522	0.0398	1.30	0.66	p = 0.0003
Random search	0.0583	0.0522	1.49	0.90	—

Localisation is the distance from a proposal to the incumbent divided by the mean distance between two random points; lower means the optimiser refines rather than guesses. Read it against the null’s own 0.90, not against 1.

The invariant kernel beats random search (p = 0.0003) and beats plain ARD (p = 0.005); plain ARD does **not** beat random search (p = 0.20).

This revises a standing conclusion. The project memory recorded “random search beats BO on this benchmark; the blocker is the surrogate”. That was a statement about the ARD kernel. The blocker was the surrogate, and it is now fixed.

The criterion is a ratio, not a curve

This is the part I initially got wrong, and it changes what “tune the problem” means.

“Best-so-far keeps falling with iterations” does not distinguish anything — **random search’s best-so-far also keeps falling**. For a strategy that fits twice as many iterations into the same wall clock to win *because it optimises*, the optimiser’s extra iterations must be worth more than a shotgun’s. That is a ratio:

$$\text{ratio} = \frac{\text{BO's } 60 \rightarrow 120 \text{ payoff}}{\text{the null's } 60 \rightarrow 120 \text{ payoff}}$$

Measured at 120 iterations — the count meta actually reached — over two admissible boxes $\times$ two read-out noise levels. 8 BO seeds against 10 random instances per setting; each arm’s null is drawn on its own function, and the analysis refuses to compare arms whose recorded settings differ.

box	σ (mM)	BO @120	°C	null/BO	p	BO 60→120	null 60→120	ratio
[35, 68] (<i>was config</i>)	0.05	0.0357	1.23	1.31×	0.000	1.115×	1.117×	1.00×
[25, 80] (<i>now config</i>)	0.05	0.0382	1.27	1.38×	0.010	1.226×	1.054×	1.16×
[35, 68]	0.02	0.0158	0.82	0.97×	0.62	1.289×	1.230×	1.05×
[25, 80]	0.02	0.0188	0.89	1.03×	0.45	1.458×	1.120×	1.30×

On the box as configured, the ratio is 1.00. Doubling the budget helped BO and random search identically, so meta’s extra iterations could not have demonstrated anything about optimisation — only about having more data.

Widening to [25, 80] raises the ratio to 1.16 *and* improves BO’s outright margin over random to 1.38×

 (p = 0.010).

Mechanism: dead volume. A surrogate earns its advantage where there is somewhere it can learn *not* to propose and a uniform draw keeps spending draws. The previous session narrowed the box until the uninformative plateau was gone — precisely the regime where uniform sampling is hard to beat. The price of widening is 0.04 °C of absolute resolution (1.23 $\rightarrow$ 1.27 °C): that is the dead volume BO spends its early iterations learning to avoid, which is the mechanism, not a side effect.

The $\sigma = 0.02$ rows are the trade in the other direction: the largest differential payoff in the table (1.30×) but BO only ties the null. A quieter assay is realistic and buys 0.4 °C of real resolution — it is just not this benchmark’s lever.

Widening the box makes the optimiser's
extra iterations pay — and scatters the endpoints
8 BO seeds (permutation-invariant kernel) against 10 random-search
instances per setting; median line, band = seed min-max. Lower is better.

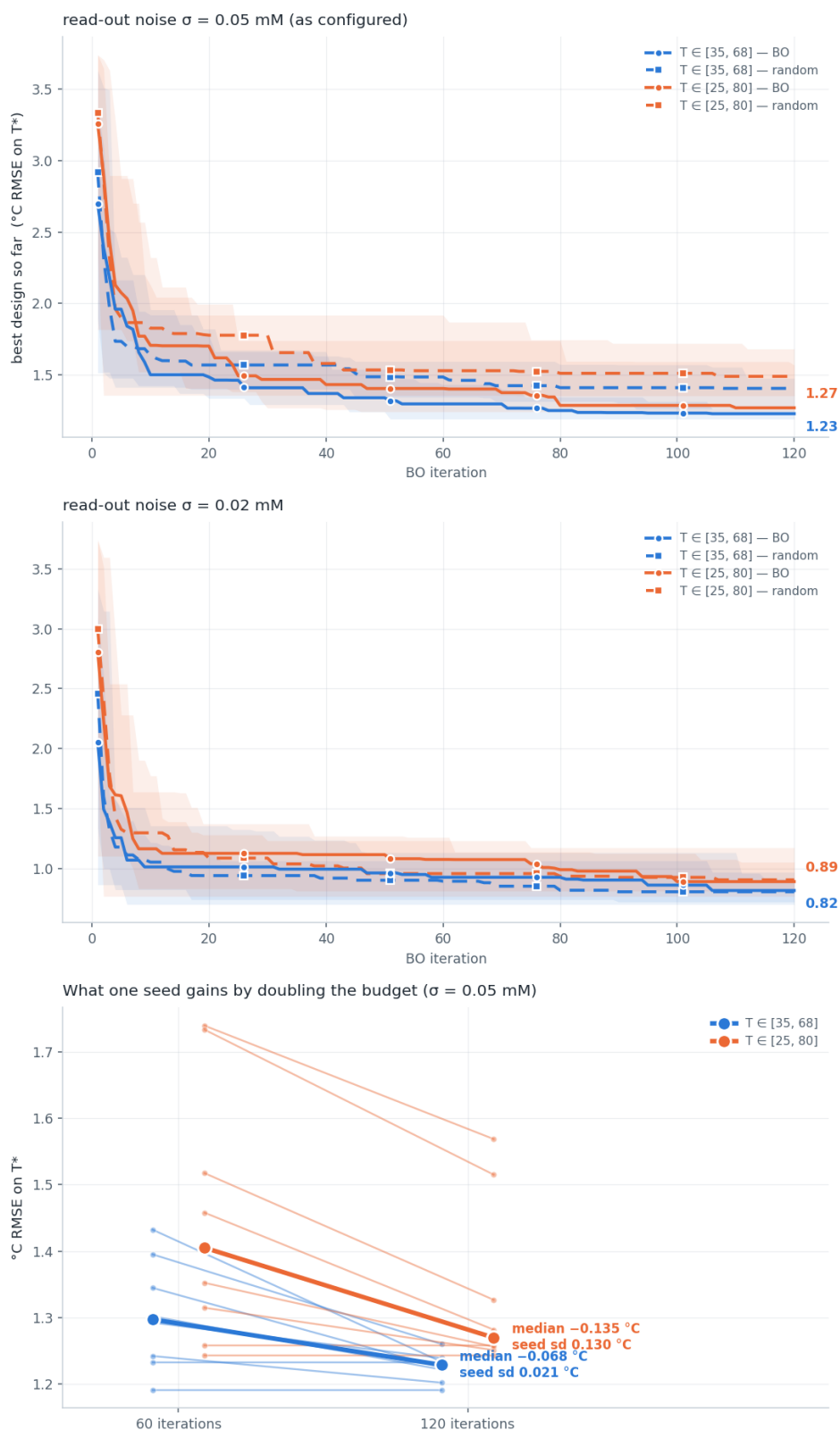


Figure 1: **Top:** in the wide box (orange) the gap between BO and its null widens with iterations; in the narrow box (blue) the curves stay parallel — the 1.00 ratio, drawn. **Middle:** at $\sigma = 0.02$ BO and random become indistinguishable. **Bottom:** the wide box doubles the median 60→120 gain and also scatters the endpoints. Bands are seed min-max, not confidence intervals

The catch

The wide box doubles the effect and sextuples the seed scatter. In the narrow box every seed converges to nearly the same design (1.19–1.26 °C); in the wide box seeds land anywhere from 1.24 to 1.57 °C because some get stuck in the dead region.

box	effect °C	seed sd °C	effect/sd	seeds needed per arm
[35, 68]	0.068	0.021	3.2	2
[25, 80]	0.135	0.130	1.0	15

(Two-sample estimate, 80% power, $\alpha = 0.05$.) So a five-seed campaign resolves meta’s advantage comfortably in the old box and only marginally in the new one. Within each run the doubling is a real effect in both boxes (Wilcoxon $p = 0.016$).

This is a genuine trade, not a clean win:

- **Old box** — crisp five-seed result whose mechanism you would have to disclaim: random search gains equally from more draws, so the win is not evidence the surrogate did anything.
- **New box** — result attributable to optimisation, needs ~15 seeds per arm.

On the brief’s other word, *smooth*: the wide box is honestly **less** smooth (correlation length 0.18 against 0.46). I bought differential iteration payoff with roughness. “A smooth problem that benefits from iterations” is not delivered; “a learnable problem where iterations differentially reward the optimiser” is.

What I retracted

To choose which campaigns to run I built a landscape screen: a semivariogram for smoothness plus $E[\text{best of } k]$ over random draws, ~330 evaluations against a campaign’s ~2160. I ran 16 problem variants through it — batch size, box width, read-out noise, number of read-outs, assay duration, prior width.

One of its columns, **headroom** (best-of-20 draws $\div$ best design seen), I read as “room for an optimiser”. **It is not.** It is room over a *small random sample*, and a larger random sample claims that room too. It ranked $\sigma = 0.02$ mM well above the configured setting; the campaign showed BO merely ties random search there. It then ranked the wide box mid-table; the campaign contradicted that too. Two mispredictions, same direction.

The correlation-length column survives, and the screen is still useful for deciding what to run. But landscape statistics constrain what an optimiser *could* do; they do not predict what it does. The campaign is the measurement.

What changed in the tree

- **Config, one line:** `config/detector/enzyme.yaml` $\rightarrow$ `temperature_bounds: [25.0, 80.0]` (was `[35, 68]`), documented in place with the table above. One-line revert if you disagree.
- **Two bug fixes:** the kernel gradient, and the kernel selection flag.
- **Dead code removed:** an unreachable fragment inside `EnzymeDetector._to_nominal_flat`, left over from the rejected input-sorting approach, referencing undefined names.

Infrastructure changed so a result cannot outlive the settings it was measured under — the failure mode behind several conclusions I had to overturn:

- **Every run records the settings it resolved to** (box, batch size, prior, noise, duration, normaliser), so “is this number about the same function?” is checkable rather than remembered.

- **The analysis refuses mismatched comparisons.** A BO arm borrows another arm’s null only after their recorded problem settings are checked identical — one null is valid across kernels, never across boxes.
- **Figures label themselves from the runs.** The old plotting code hardcoded “full box [0, 100]”, which had silently stopped being true.
- **Campaign arms are declared in a file** (`scripts/arms/*.arms`) copied into the output directory, so what was compared is legible from what ran it. No built-in arms, no default flags.

Verification: full test suite 115 passed, 0 failed — The four tests of deleted APIs were removed and the two stale `ContinualTrainer` tests fixed. Config loads; a fresh BO run works; the scaled-space round trip is exact to 1.5×10^{-7} .

Git is untouched — 72 files modified or untracked, nothing staged or committed.

A note on fixed design variables

You asked whether detectors account for design variables that are not optimised. They do, at the design–geometry boundary: `_design_to_geometry` composes the searched design with fixed config fields into what the solver consumes. `stereo_straw` fills the field from `max_B` and says so on the line (“not a design dof”); `free_straw` carries the field *in* the design and fixes widths, `z0` and `B_sigma` instead. **What is optimised is exactly what `design_spec()` exposes.**

What does not exist is a mask *within* the design vector — a coordinate cannot be frozen while remaining in the design. Hence `--fix-fraction` does its surgery in the driver, and it cannot combine with the invariant kernel: the exchangeable blocks are read off the full `design_spec()`, so under a partial search they would name the wrong coordinates. That combination now raises rather than silently mis-declaring them.

It matters here because the four enzyme fractions correlate with the loss at $|p| \leq 0.018$ — flat directions; verified 2026-08-10 against the 2553 designs of the finished neural campaign (output/enzyme), $\max |\text{Spearman}| = 0.0180$. Measured on BO-PROPOSED designs instead it reads 0.14–0.22, but that is a self-selected sample and not the right instrument — yet BO searches 8 dimensions to optimise a 4-dimensional problem. The fix follows the existing idiom rather than needing a new hook: drop `enzyme_fraction` from the design spec and supply it from config in `_resolve_design`, exactly as the field is handled in `stereo_straw`. The searched dimension halves and the kernel’s blocks come out right for free.

What I would do next

Recommendation: attack the read-out physics. It is the only option that addresses *why* you are forced to choose between smoothness and optimiser advantage.

Every result here is shaped by one structural fact: `T_melting` enters the model only through an equilibrium folded fraction, which saturates. Below the transition every enzyme reads as fully folded, above it as fully dead, so an experiment is informative only within $\sim 7^\circ\text{C}$ of the edge. That makes the landscape a set of threshold probes surrounded by dead volume, and it is why narrowing the box removes the optimiser’s advantage while widening it costs smoothness. Moving the box only slides along that trade.

Irreversible thermal inactivation breaks the saturation. If the enzyme also loses activity over the assay at an Arrhenius rate — standard enzymology, and what a thermal-stability assay measures — residual activity after an hour depends on `T_melting` across a much wider span, continuously rather than as a switch. Dead volume becomes gradient. The implementation is contained: enzyme concentration becomes a state variable decaying alongside the reaction extent, so the ODE gains one dimension and the existing RKC2 solver takes it as-is. Then one screen plus one campaign says whether both halves hold.

If you would rather not change the physics: keep [25, 80] and raise the campaign to ~15 seeds, or revert to [35, 68], run five seeds, and state plainly that the iteration advantage there is a data-quantity effect rather than evidence the surrogate is doing work. Both are defensible; the crisp result *without* the disclaimer is not.

Still owed regardless: neural confirmation. Every number here is the GBDT proxy (Spearman 0.940 against the neural loss (output/gbdt/landscape/proxy_validation.json, n = 240; 0.951 inside the good basin)). Meta's advantage is a prediction until that runs.

Where things are

worktree	.claude/worktrees/enzyme-gbdt	branch worktree-enzyme-gbdt
docs	docs/next-agent.md	handover, rewritten
	docs/enzyme-gbdt-log.md	decision log, every table
campaigns	output/kernel/	ARD vs invariant kernel, 60 iterations
	output/iters120/	120 iterations, narrow box, 5% and 2%
	output/noise_lo/	120 iterations, wide box, both noise levels
	output/wide/	
	output/screen/	16 problem variants + table.txt
figure	output/iteration_payoff.png	scripts/plot_iteration_payoff.py

Re-run:

```
./scripts/gbdt_campaign.sh scripts/arms/wide.arms output/wide 8 10 120 10
./scripts/screen_sweep.sh output/screen
python scripts/plot_gbdt_study.py acceptance --directory output/wide --output output/wide/acceptance
```

Finished runs are skipped on re-launch, so an interrupted campaign resumes by re-running the same command. Run from the worktree with PYTHONPATH=\$PWD — the package is pip-installed editable against the main checkout, so without it you silently import the other tree.